resmon — Research Monitor

build pending

license see LICENSE

platform desktop (Electron)

status phase 2

Overview

resmon is an automated, customizable literature surveillance platform that monitors open-access scholarly repositories, aggregates newly published papers against user-defined criteria, and compiles chronological reading reports. By default it functions as a streamlined metadata and abstract extractor; an optional AI-powered pipeline can generate concise, customized summaries of abstracts, methodologies,

results, and discussions. The desktop application surfaces this capability through a dashboard of active routines and recent activity, manual Deep Dive and Deep Sweep runs, scheduled Automated Deep Sweeps, a calendar of upcoming and past executions, and a local-first architecture that optionally synchronizes reports and credentials to a signed-in cloud account.

Key Features

- **Multi-repository ingestion across 16 open-access sources** — unified metadata normalization over arXiv, bioRxiv, medRxiv, CORE, CrossRef, DBLP, DOAJ, EuropePMC, HAL, IEEE Xplore, NASA ADS, OpenAlex, PLOS, PubMed, Semantic Scholar, and Springer Nature. Each client enforces per-source rate limiting, exponential backoff, and graceful degradation when a single source fails mid-sweep.
- **Three operational modes:**
  - *Targeted Deep Dive* — a focused, manual query against a single repository within a defined date range, with support for an ephemeral per-execution API key that never persists to disk.
  - *Broad Deep Sweep* — a cross-repository manual query that applies Deep Dive parameters across every selected repository in parallel.
  - *Automated Deep Sweep (Routine)* — a background-scheduled Deep Sweep that runs on a cron expression, emits progress events, and optionally triggers email notifications and cloud uploads on completion.
- **AI-powered summarization** — optional dual-path LLM integration covering remote commercial APIs (BYOK) and local/open-weight model inference, with token-aware chunking and customizable summarization prompts applied to abstracts, methodologies, results, and discussions. API keys are stored per provider in the OS keyring (one slot for each of `anthropic`, `openai`, `google`, `xai`, `meta`, `deepseek`, `alibaba`, `local`, and `custom`), and every per-execution AI override panel on Deep Dive, Deep Sweep, and Routines exposes the full Settings → AI control set (Provider, Model, Length, Tone, Temperature, Extraction Goals) with per-field merge semantics so a single override never clobbers persisted defaults.
- **Cross-platform desktop notifications** — routine and manual completions raise a native OS notification on macOS, Linux, and Windows. The notification dispatcher is invoked both from the foreground app and from the headless `resmon-daemon`, so completions fire even when the Electron UI is closed.
- **Calendar scheduling** — a calendar view of scheduled routines and historical executions, driven by the scheduler service and the `/api/calendar/events` endpoint.
- **Email notifications** — per-routine and global SMTP-based notifications on routine completion, including attachment of the execution bundle produced by the shared export pipeline.
- **Cloud backup and hybrid execution** — optional sign-in to a resmon cloud account for envelope-encrypted credential storage, report synchronization, and a merged local/cloud executions view.
- **Configuration export and import** — serialize any Deep Dive, Deep Sweep, or routine configuration to JSON and re-import it on the same or another device for reproducible surveillance setups.

- **Local-first storage and logging** — SQLite-backed state, per-execution log files, and a configurable export directory for report and artifact bundles; no credentials or data leave the machine unless the user opts in to cloud sync.

Supported Repositories

The table below lists the 16 active sources registered in the repository catalog (`/api/repositories/catalog`). "API key" indicates whether a key is required to query the source from resmon; rate limits are the client-side ceilings enforced by each API client.

Repository	API Type	API Key	Rate Limit (resmon)	Discipline Coverage
arXiv	REST (Atom XML)	Not required	0.33 req/s (1 per 3 s)	Physics, Math, CS, Quant-bio, Stats, EE, Econ
bioRxiv	REST (JSON)	Not required	2.0 req/s	Life-sciences preprints
medRxiv	REST (JSON, shared client with bioRxiv)	Not required	2.0 req/s	Health-sciences preprints
CORE	REST (JSON)	Required (Bearer)	5.0 req/s	Multi-disciplinary open access
CrossRef	REST (JSON)	Not required	10.0 req/s (polite pool)	All disciplines (DOI-indexed)
DBLP	REST (JSON)	Not required	2.0 req/s	Computer science
DOAJ	REST (JSON)	Not required	5.0 req/s	All disciplines (OA journals)
EuropePMC	REST (JSON)	Not required	5.0 req/s	Biomedicine, Life sciences
HAL	REST (Solr JSON)	Not required	2.0 req/s	All disciplines (French-leaning)
IEEE Xplore	REST (JSON)	Required (API key)	0.2 req/s (1 per 5 s)	Electrical engineering, CS, Electronics

Repository	API Type	API Key	Rate Limit (resmon)	Discipline Coverage
NASA ADS	REST (Solr JSON)	Required (Bearer)	1.0 req/s (≈5000/day cap)	Astronomy, Astrophysics, Planetary science
OpenAlex	REST (JSON)	Not required	10.0 req/s (polite pool via mailto)	All disciplines
PLOS	REST (Solr JSON)	Not required	5.0 req/s	Biology, Medicine, Natural sciences (PLOS journals)
PubMed / NCBI E-utilities	REST (XML)	Optional (raises limit)	3.0 req/s keyless, 10.0 req/s with key	Biomedicine
Semantic Scholar	REST (JSON)	Optional (recommended)	0.33 req/s (1 per 3 s)	All disciplines (strong CS, biomed)
Springer Nature	REST (JSON Meta API)	Required (query-param key)	5.0 req/s (≈5000/day cap)	STM, Humanities, Social sciences

Sources previously evaluated but excluded from the active catalog (SSRN, RePEc/IDEAS) are documented in `.ai:/prep/repos.md` and are not queried at runtime.

Installation

Prerequisites

resmon is a hybrid Python + Electron application, so both runtimes must be available on the host machine before installation.

- **Python 3.10 or newer** — required by the FastAPI backend (`resmon_scripts/resmon.py`) and its dependencies. Verify with `python3 --version` .
- **Node.js 18 or newer** — required to build the React renderer with Webpack and to run the Electron shell. Verify with `node --version` .
- **npm 9 or newer** — bundled with recent Node.js releases; used to install frontend dependencies and invoke the build/start scripts. Verify with `npm --version` .
- **Git** — required to clone the repository.

- **Platform** — macOS, Linux, or Windows. The packaged desktop app is built with `electron-builder` and the headless-daemon split supports launchd (macOS), `systemd --user` (Linux), and Task Scheduler (Windows).

Backend Setup

Clone the repository, create an isolated Python virtual environment at the project root, and install the pinned backend dependencies:

```
git clone <repository-url> resmon
cd resmon

python3 -m venv .venv
source .venv/bin/activate           # macOS / Linux
# .venv\Scripts\activate.bat        # Windows (cmd)
# .venv\Scripts\Activate.ps1        # Windows (PowerShell)

pip install --upgrade pip
pip install -r requirements.txt
```

The backend reads its SQLite database from `resmon.db` at the project root on first launch and creates the `resmon_reports/` subtree (`markdowns/` , `pdfs/` , `figures/` , `latex/` , `logs/` , `info_docs/`) automatically.

Frontend Setup

Install the Node dependencies and produce the production renderer and Electron-main bundles:

```
cd resmon_scripts/frontend
npm install
npm run build
```

`npm run build` chains two steps: `webpack --mode production` compiles the React renderer into `resmon_scripts/frontend/dist/renderer/` , and `tsc --project tsconfig.electron.json` transpiles the Electron main process into `resmon_scripts/frontend/dist/electron/` . Re-run `npm run build` whenever frontend or Electron-main source files change. During active development, `npm run dev:renderer` runs Webpack in watch mode against the renderer sources only.

Launching the Application

The desktop experience is a three-process composition: the Electron main process spawns the Python FastAPI backend as a child process (or attaches to an already-running headless daemon), hosts the renderer window, and bridges OS-level capabilities through a contextBridge-exposed `window.resmonAPI` surface. All communication between the renderer and the backend is `fetch` -over-localhost JSON plus Server-Sent Events for live progress — there is no direct IPC from the renderer to Python.

To launch the app for local development or everyday use:

```
# From the project root, with the Python virtual environment active:
source .venv/bin/activate

# Launch Electron — it will spawn the Python backend and open the main window.
cd resmon_scripts/frontend
npm start
```

`npm start` runs `npm run build` followed by `electron .` , which loads `dist/electron/main.js` and opens the renderer at the `/` route. The backend binds to `127.0.0.1:8742` by default; the renderer discovers the port through `window.resmonAPI.getBackendPort()` .

The renderer uses a `HashRouter` (the packaged app is loaded from `file://` , where push-state history is unreliable), so every page is reachable under a `#/…` fragment:

Path	Page
`#/`	Dashboard
`#/dive`	Deep Dive
`#/sweep`	Deep Sweep
`#/routines`	Routines
`#/calendar`	Calendar
`#/results`	Results & Logs
`#/configurations`	Configurations
`#/monitor`	Monitor
`#/repositories`	Repositories & API Keys
`#/settings/*`	Settings (nested router)

A `Sidebar` + `Header` + `MainContent` layout wraps every route, and a `FloatingWidget` monitor overlay sits outside `MainContent` so it survives route transitions and continues to pulse while any execution is running.

Quick Start Guide

The following walkthroughs cover the three core workflows. Each flow assumes the app is running and that any required API keys for keyed repositories (CORE, IEEE Xplore, NASA ADS, Springer Nature) are either stored under **Repositories & API Keys** or provided as ephemeral per-execution keys.

Run a Deep Dive

A Deep Dive runs a targeted one-off query against a **single** repository.

1. Open the **Deep Dive** page at `#/dive`.
2. In the repository selector (single-select), choose one source.
3. Optionally restrict the date range with **Date Range** — note that the date range is intentionally **not** saved as part of a configuration and must be picked fresh for every run.
4. Enter one or more keywords in the chip-style **Keywords** input; keywords are joined with spaces to form the query.
5. Adjust the **Max Results** slider (range 10–500, step 10, default 100).
6. Optionally toggle **Enable AI Summarization**. When enabled, the *Override AI settings for this run* disclosure offers per-execution overrides for length, tone, and model; empty overrides fall back to the app defaults.
7. If the selected repository requires a credential that is not already stored in the OS keyring, enter an ephemeral key in the inline **Key Status** panel. Ephemeral keys are sent only in the request body and are never persisted.
8. Click **Run Deep Dive**. The page sends `POST /api/search/dive`, registers the returned execution id with `ExecutionContext`, and the floating widget begins streaming progress. When the execution reaches a terminal state, an **Execution #N** result card is rendered inline with counts and a **View Report** link to `#/results?exec=<id>`.

Run a Deep Sweep

A Deep Sweep runs a broad one-off query across **multiple** repositories in parallel, then normalizes and de-duplicates the combined result set by DOI and by title + first author.

1. Open the **Deep Sweep** page at `#/sweep`.
2. In the repository selector (multi-select), choose every source to include.
3. Enter one or more keywords; the same query string is issued to every selected repository.

4. Optionally restrict the date range (again, not persisted by *Save Configuration*).
5. Adjust the **Max Results** slider — the cap applies **per repository**, so the merged result set can be up to `repositories.length × max_results` before de-duplication.
6. Optionally toggle **Enable AI Summarization** and its per-execution overrides.
7. For each selected repository that requires a credential, enter an ephemeral key in the inline Key Status panel if the key is not already stored in the keyring.
8. Click **Run Deep Sweep**. The page sends `POST /api/search/sweep` and registers the execution with `ExecutionContext`. The backend first calls an admission controller; if the concurrent-execution cap is reached, the server returns HTTP `429` with a `Retry-After: 5` header and a message that points to **Settings** → **Advanced** — this surfaces verbatim as the form error.

Create and Activate a Routine

A Routine is a scheduled Automated Deep Sweep: the same multi-repository pipeline, bound to a saved configuration and a 5-field cron expression.

1. Open the **Routines** page at `#/routines` and click **New Routine** to open the create modal.
2. Enter a **Name** and choose a **Cron expression** (5-field `m h dom mon dow`, default `0 8 * * *`). Any configuration-related fields can be pre-populated by loading a saved `routine` configuration through the inline **ConfigLoader** — this restores everything except the date range.
3. Select repositories, enter keywords, and set an optional date range and a per-repository **Max Results** cap.
4. Set the per-routine flags as needed:
   - **AI** — enable AI summarization for each fire.
   - **Email** — email the report on completion (the global SMTP settings must be configured under **Settings** → **Email**).
   - **Results-in-Email** — include the AI summary inline in the notification email.
   - **Notify-on-Completion** — raise a desktop notification when a fire finishes.
5. Choose an **Execution location**: **Local** (fired by the APScheduler thread inside the local resmon daemon) or **Cloud** (fired by the `resmon-cloud` scheduler). The Cloud radio is disabled unless cloud sync is enabled and the user is signed in.
6. Submit the modal. The routine is persisted via `POST /api/routines` (local) or `POST /api/v2/routines` (cloud), and APScheduler registers a job under the routine's id.
7. Manage the routine from its row in the unified routines table: **Activate/Deactivate** preserves the DB row while adding or removing the APScheduler job; the inline **Email**, **AI**, and **Notify** column toggles patch a single flag each; **Move to Cloud** / **Move to Local** performs a destination-first create followed by a source delete; and if a fire is currently running, a **Cancel Run** button appears on the row and stops the live execution.

Architecture Overview

resmon is a local-first desktop application composed of three cooperating processes: an Electron main process, a Python FastAPI backend, and an optional resmon-cloud microservice. The Electron main process launches the Python backend as a child process (or attaches to an already-running headless daemon managed by `daemon.py` + `service_manager.py`) and hosts the React renderer window. The renderer never calls Python directly — it talks to the backend exclusively over `fetch` -based JSON on `http://127.0.0.1:<port>` (default `8742` , discovered via `window.resmonAPI.getBackendPort()`) and over Server-Sent Events for live progress.

Backend — FastAPI + SQLite

The backend is a single FastAPI application constructed at module load in `resmon_scripts/resmon.py` . A shared `sqlite3.Connection` backs every request, the database path defaults to `resmon.db` at the project root, and the schema is owned by `implementation_scripts/database.py` with a version-tracked migration path on startup. Two ASGI middlewares wrap the app: a custom `PrivateNetworkMiddleware` that injects `Access-Control-Allow-Private-Network: true` so Chromium's Private Network Access policy permits the `file://` renderer to reach loopback, and `CORSMiddleware` with permissive origins (safe because the server binds to `127.0.0.1` only). All SQL is parameterized; all credentials flow through a single `credential_manager.py` module that owns OS-keyring access.

The core pipeline is `SweepEngine` (`implementation_scripts/sweep_engine.py`), which orchestrates query → normalize → dedup → link → report → summarize → finalize for both manual and routine-fired runs. Per-source API clients (16 repositories) live under `implementation_scripts/api_*.py` and are registered through `api_registry.py` . Results are normalized by `normalizer.py` , deduplicated by DOI and by (title, first author), and rendered by `report_generator.py` into Markdown, with optional PDF and LaTeX exports through `report_exporter.py` .

Frontend — Electron + React

The renderer is a React 18 + TypeScript single-page application bundled with Webpack and served into an Electron window (`electron/main.ts` , `dist/electron/main.js`). Routing uses `react-router-dom` `HashRouter` because the packaged app is loaded from `file://` where push-state history is unreliable. Two React contexts wrap every route: `AuthProvider` for resmon-cloud identity (access tokens held only in a module-scoped `authStore` , refresh tokens in the OS keychain) and `ExecutionProvider` for live multi-execution state. All HTTP access is centralized in four API-client wrappers (`api/client.ts` , `api/cloudClient.ts` , `api/repositories.ts` , `api/authStore.ts`), each of which sets `Cache-Control: no-store` and unwraps FastAPI's `{detail: "..."}` error envelope into a single readable message. A `FloatingWidget` overlay is mounted outside `MainContent` so it survives route transitions and continues to pulse for any running execution.

LLM Integration — BYOK Remote and Local (ollama)

AI summarization is optional and fully bring-your-own-key. `implementation_scripts/llm_factory.py` (`build_llm_client_from_settings`) reads the persisted `ai_*` settings and returns either a `RemoteLLMClient` (OpenAI, Anthropic, Google, xAI, Meta, DeepSeek, Alibaba, or a user-defined custom provider), a `LocalLLMClient` backed by a local ollama instance over its REST API (`implementation_scripts/llm_local.py`), or `None` when the provider is unset — "AI unconfigured" is a silent no-op, never an exception. The custom-provider base URL is validated to be HTTPS unless the host is loopback, both in the Settings → AI panel and in the factory. Token-aware chunking and provider-agnostic summarization live in `summarizer.py` with prompt scaffolding in `prompt_templates.py` . API keys are never logged, never included in exception messages, and never returned by any GET endpoint; key lookup honors an ephemeral per-execution stack that takes precedence over the keyring value.

Scheduler — APScheduler with SQLAlchemy Job Store

Routine firing is driven by `ResmonScheduler` (`implementation_scripts/scheduler.py`), a thin wrapper over APScheduler's `BackgroundScheduler` with a `SQLAlchemyJobStore` backed by `sqlite:///resmon.db` , so jobs persist across app restarts. Cron parsing is strict 5-field (`minute hour day month day_of_week`). The scheduler is decoupled from FastAPI via a dispatcher indirection: `set_dispatcher(fn)` installs the callback invoked on each fire, and the APScheduler worker thread can never raise because a missing dispatcher is logged and returned. On startup, `_init_scheduler_on_startup` re-registers every active routine from `get_routines(conn)` ; on shutdown, the scheduler is gracefully stopped.

Admission Controller

`implementation_scripts/admission.py` (`ExecutionAdmissionController`) is the single global gate on concurrent executions. `max_concurrent` is clamped to `[1, 8]` (default `3`) and the routine-fire queue is clamped to `[1, 64]` (default `16`). **Manual** admission is reject-or-pass: if no slot is free, REST endpoints raise HTTP `429` with a `Retry-After: 5` header that the Deep Dive / Deep Sweep forms surface verbatim. **Routine** admission falls through to a bounded FIFO queue that drains as slots free; overflow past `queue_limit` is dropped with a logged error. Live limits are reloaded from the `settings` table at startup and updated through `PUT /api/settings/execution` (Settings → Advanced panel).

SSE Progress Stream

Live progress is streamed through `ProgressStore` (`implementation_scripts/progress.py`), a thread-safe in-memory bus between pipeline worker threads and the SSE endpoint. Each `exec_id` owns its own event list, lock, cancel-flag `threading.Event` , and completion boolean. The backend exposes three surfaces on top of it:

- `GET /api/executions/active` — a cheap `{active_ids: [...]}` payload polled by `ExecutionContext` every 3 seconds as a safety net that attaches background-initiated runs and detects dropouts.
- `GET /api/executions/{id}/progress/events` — the full event list, used by the 1-second per-execution poller that drives `ExecutionContext` and thus the Monitor page and FloatingWidget.
- `GET /api/executions/{id}/progress/stream` — a true SSE endpoint emitting `event: progress` frames with `id: <cursor>` headers, `~300 ms` heartbeats (`: heartbeat\n\n`), `last_event_id` resumption, and a post-terminal drain that replays any persisted-but-unread events before closing.

Standard SSE headers are set (`Cache-Control: no-cache` , `Connection: keep-alive` , `X-Accel-Buffering: no`) and `PrivateNetworkMiddleware` does not buffer the stream. After an execution ends, buffered events are persisted into the `execution_progress` table so the Monitor and Results pages can reconstruct the stream after a process restart. Cancellation is cooperative: `POST /api/executions/{id}/cancel` sets the cancel flag, the pipeline's 2-second heartbeat notices, a partial report is flushed, and the execution finalizes as `cancelled` .

Optional Cloud Sync

Two distinct cloud surfaces coexist and must not be conflated:

1. **Google Drive artifact backup** (`implementation_scripts/cloud_storage.py`) — least-privilege `drive.file` OAuth 2.0 scope, token stored in the OS keyring, uploads triggered by `SweepEngine._maybe_auto_backup` when the `cloud_auto_backup` setting is on. Configured under Settings → Cloud Storage.
2. **resmon-cloud account + multi-device sync** — a separate microservice backing envelope-encrypted per-user credentials (per-user DEK wrapped by a KMS-held KEK), JWKS-verified JWTs, row-level security, signed-URL artifact fetches, and per-user rate limiting. The desktop consumes it through `AuthContext` , `api/cloudClient.ts` , and `useCloudSync` , which polls `GET /api/v2/sync?since=<version>` on a 60-second interval (also on focus-gain), drains `has_more` chains up to `MAX_PAGES_PER_TICK = 50` , and POSTs pages into the local daemon's `/api/cloud-sync/ingest` . Cloud-sync errors never propagate into the render tree — they are surfaced through `state.lastError` only.

Technology Stack

Category	Component	Purpose
Backend	Python 3.10+, FastAPI 0.135, Starlette 1.0, Uvicorn 0.44, Pydantic 2.12	HTTP server, request validation, ASGI runtime

Category	Component	Purpose
Backend	httpx 0.28, lxml 6.0, beautifulsoup4 4.14	Repository API clients, XML/HTML parsing
Backend	cryptography 46, keyring 25.7	OS-keyring credential storage, envelope encryption helpers
Backend	google-api-python-client 2.194, google-auth-oauthlib 1.3	Google Drive artifact backup (OAuth 2.0, `drive.file` scope)
Backend	nltk 3.9, tiktoken 0.12	Text normalization and token-aware chunking for summarization
Frontend	Electron 41, Node.js 18+, npm 9+	Desktop shell, main-process bridging
Frontend	React 19, React Router 7 (`HashRouter`), TypeScript 6	Renderer SPA and routing
Frontend	Webpack 5, ts-loader 9, css-loader 7, style-loader 4, html-webpack-plugin 5	Build pipeline
Frontend	Tailwind CSS 4	Utility-first styling
Frontend	FullCalendar 6 (`@fullcalendar/react` , `daygrid` , `timegrid` , `interaction`)	Calendar page rendering
Frontend	`electron-builder` 26	Packaged-app distribution
Database	SQLite 3 (stdlib `sqlite3`), SQLAlchemy 2.0	Local state, schema migrations, APScheduler job store
LLM — Remote (BYOK)	`openai` 2.31, `anthropic` 0.95, plus provider-agnostic `httpx` clients for Google, xAI, Meta, DeepSeek, Alibaba, and custom HTTPS endpoints	Remote summarization backends
LLM — Local	ollama REST API (`/api/generate` , `/api/tags`) via `llm_local.py`	On-device open-weight model inference (Llama, Gemma, Qwen, etc.)

Category	Component	Purpose
Scheduling	APScheduler 3.11 (`BackgroundScheduler` + `SQLAlchemyJobStore`)	Routine cron jobs, persistent across restarts
Service integration	launchd (macOS), `systemd --user` (Linux), Task Scheduler (Windows) via `service_manager.py`	Optional headless-daemon OS service
Email	stdlib `smtplib` (via `email_notifier.py` + `email_sender.py`), `python-multipart` 0.0.26	Transactional SMTP notifications and attachment handling
Testing	pytest 9.0, pytest-timeout 2.4	Backend unit and integration tests under `resmon_scripts/verification_scripts/`
Testing	TypeScript `tsc --noEmit` (`npm run typecheck`)	Frontend type checking

Project Structure

```
resmon/
├── README.md                       # This file.
├── LICENSE                         # MIT license.
├── requirements.txt                # Pinned Python dependencies.
├── credentials.json                # Google OAuth 2.0 client secrets (user-supplied, gitig
├── resmon.db                       # SQLite database (created on first launch).
│
├── resmon_scripts/                 # All application source code.
│   ├── resmon.py                   # FastAPI app entrypoint (routes, startup/shutdown hook
│   │
│   ├── implementation_scripts/     # Backend modules.
│   │   ├── admission.py              # Concurrent-execution admission controller.
│   │   ├── ai_models.py              # Provider model-catalog probing.
│   │   ├── api_*.py                  # Per-repository API clients (16 active sources).
│   │   ├── api_base.py               # Shared rate limiter + HTTP client base class.
│   │   ├── api_registry.py           # Slug → client dispatch table.
│   │   ├── citation_graph.py         # Citation and context graphing.
│   │   ├── cloud_storage.py          # Google Drive backup (drive.file scope).
│   │   ├── config.py                 # APP_NAME, paths, defaults.
│   │   ├── config_manager.py         # Configuration JSON export/import.
│   │   ├── credential_manager.py     # OS-keyring credential vault + ephemeral stack.
│   │   ├── daemon.py                 # Headless-daemon lock file, state dir, attach/spawn
│   │   ├── database.py               # Schema, migrations, parameterized queries.
│   │   ├── email_notifier.py         # Email templating and dispatch.
│   │   ├── email_sender.py           # SMTP transport.
│   │   ├── llm_factory.py            # Provider whitelist + client construction.
│   │   ├── llm_local.py              # ollama REST client.
│   │   ├── llm_remote.py             # Remote-BYOK client (OpenAI, Anthropic, etc.).
│   │   ├── logger.py                 # Rotating app logger + per-execution TaskLogger.
│   │   ├── normalizer.py             # Cross-source metadata normalization + dedup.
│   │   ├── progress.py               # ProgressStore (SSE/poll event bus + cancel flag).
│   │   ├── prompt_templates.py       # Summarization prompt scaffolding.
│   │   ├── repo_catalog.py           # Repository metadata catalog.
│   │   ├── report_generator.py       # Markdown report composition.
│   │   ├── report_exporter.py        # PDF / LaTeX export pipeline.
│   │   ├── scheduler.py              # ResmonScheduler (APScheduler wrapper).
│   │   ├── service_manager.py        # launchd / systemd / Task Scheduler integration.
│   │   ├── summarizer.py             # Token-aware chunking + provider-agnostic summarizat
│   │   ├── sweep_engine.py           # End-to-end query → dedup → report → summarize pipel
│   │   ├── utils.py                  # Shared helpers.
│   │   └── assets/                   # Static backend assets (templates, figures).
│   │
```

```
│   ├── frontend/                  # Electron + React renderer.
│   │   ├── package.json             # Node dependencies and build scripts.
│   │   ├── webpack.config.js        # Renderer bundle config.
│   │   ├── tsconfig.json            # Renderer TypeScript config.
│   │   ├── tsconfig.electron.json   # Electron-main TypeScript config.
│   │   ├── electron/                # Electron main-process sources (preload, IPC bridge)
│   │   ├── src/                     # Renderer SPA.
│   │   │   ├── index.tsx              # React root.
│   │   │   ├── App.tsx                # HashRouter + layout + providers.
│   │   │   ├── api/                   # Typed HTTP client wrappers.
│   │   │   ├── components/            # Shared components (Sidebar, Header, FloatingWidge
│   │   │   ├── context/               # AuthContext, ExecutionContext.
│   │   │   ├── hooks/                 # useExecutionsMerged, useRepoCatalog, useCloudSync
│   │   │   ├── pages/                 # One component per route (DashboardPage, …).
│   │   │   ├── styles/                # Global stylesheets.
│   │   │   ├── types/                 # Shared TypeScript types.
│   │   │   └── __tests__/             # Renderer unit tests.
│   │   └── dist/                    # Build output (renderer/, electron/).
│   │
│   ├── cloud/                     # resmon-cloud microservice sources.
│   ├── cloud_deploy/              # Cloud deployment manifests.
│   ├── service_units/             # launchd plists, systemd units, Task Scheduler XML.
│   ├── given_scripts/             # Reference scripts not part of the runtime.
│   ├── notebooks/                 # Ad-hoc development notebooks.
│   └── verification_scripts/      # pytest-based backend verification suite.
│
├── resmon_reports/                # All user-facing outputs (REPORTS_DIR).
│   ├── markdowns/                   # Per-execution Markdown reports.
│   ├── pdfs/                        # Exported PDF reports.
│   ├── latex/                       # Exported LaTeX reports.
│   ├── figures/                     # Static figures embedded in reports.
│   ├── logs/                        # resmon.log (5 MB × 3 backups) + per-execution TaskL
│   └── info_docs/                   # 11 implementation-grounded info documents.
│
├── resmon_experiments/            # Ad-hoc experimental artifacts outside the report pipe
├── resmon_printouts/              # Ad-hoc printouts / scratch outputs.
└── resmon.app/                    # Packaged macOS app bundle (electron-builder output).
```

OS-scoped state (daemon lock file, service-unit-managed logs) lives outside the repository, under `~/Library/Application Support/resmon` on macOS, `$XDG_STATE_HOME/resmon` on Linux, and `%LOCALAPPDATA%\resmon` on Windows (resolved by `daemon.state_dir()`).

Configuration

resmon persists reusable search parameters as named rows in the `configurations` table and treats them as the primary unit of reproducibility for Deep Dive, Deep Sweep, and Routine runs. A configuration is a saved bundle of parameters — repository (or repository set), keywords, per-repository result cap, AI toggle and AI settings, email toggles, storage settings, and (for routines) schedule and execution-location fields. Date ranges are deliberately **not** persisted; they are chosen fresh per run or per routine fire so a saved configuration always produces a time-current window.

Configuration Types

Every row carries a `config_type` constrained by a CHECK constraint to one of three values:

`config_type`	Produced by	Consumed by	Purpose
`manual_dive`	Deep Dive page's **Save Configuration** action	Deep Dive page's **ConfigLoader**	Reusable single-repository query preset.
`manual_sweep`	Deep Sweep page's **Save Configuration** action	Deep Sweep page's **ConfigLoader**	Reusable multi-repository query preset.
`routine`	Routines page's create/edit modal	Routines page (restores everything except the date range) and the Calendar popover	Scheduled Automated Deep Sweep bound to a cron expression.

Rows are keyed by an auto-increment integer `id` ; names are not uniquely constrained, so multiple configurations with the same name are permitted. The Configurations page at `#/configurations` opens on the **Routine Configs** tab by default and exposes a **Manual Configs** tab that shows `manual_dive` and `manual_sweep` entries together.

JSON Shape

The `parameters` column stores a JSON document whose shape matches the search page that produced it. Representative (non-exhaustive) field sets:

- **`manual_dive`** — `repository` (single slug), `keywords` , `max_results` , `ai_enabled` , optional `ai_overrides` (`length` , `tone` , `model`).
- **`manual_sweep`** — `repositories` (array of slugs), `keywords` , `max_results_per_repo` , `ai_enabled` , optional `ai_overrides` .

- **`routine`** — every `manual_sweep` field plus `cron_expression`, `execution_location` (`local` or `cloud`), `email_enabled`, `results_in_email`, `notify_on_completion`, and a `linked_routine_id` pointing at the row in the `routines` table that APScheduler registers a job under.

Deleting a configuration whose `config_type` is `routine` cascades to the linked routine row when the stored parameters contain a valid `linked_routine_id` pointing at an existing routine; the delete-confirmation dialog surfaces the cascade count before the user confirms.

Export and Import Workflow

The Configurations page supports a round-trip JSON workflow for moving presets between machines:

1. Select one or more rows via the per-row checkboxes or the select-all header checkbox.
2. Click **Export Selected** — the frontend sends `POST /api/configurations/export` with `{ ids: [...] }`; the backend writes a ZIP archive of individual JSON files through `config_manager.export_configs` and returns `{ "path": "<absolute path>" }`.
3. The success banner shows the archive path for 10 seconds; on desktop a **Reveal in Finder** (macOS) / **Reveal in File Explorer** button is rendered when the Electron preload has exposed `window.resmonAPI.revealPath`.
4. On the receiving machine, click **Import** and pick one or more `.json` files through the native file picker. Each file is validated — any file whose name does not end in `.json` short-circuits the entire batch with an inline error — and surviving files are POSTed as `multipart/form-data` to `/api/configurations/import`. The backend writes each upload to a temporary `.json` file and hands it to `config_manager.import_configs`, which validates the payload and inserts a new configuration row. The response carries `{ "imported": <n>, "errors": [] }`.
5. Click **Delete Selected** to open the confirmation dialog (which counts how many of the selected rows are routines so the cascade impact is visible). Confirming issues one `DELETE /api/configurations/{id}` per selected row; per-row failures are swallowed so the batch proceeds.

Imports are always additive: a new row is inserted with a fresh integer id. Nothing is overwritten, so round-tripping is safe to repeat.

API Reference

All REST endpoints are served by the local FastAPI daemon (`resmon_scripts/resmon.py`) bound to `127.0.0.1:8742` by default. Requests and responses are JSON unless noted; error responses use FastAPI's standard `{"detail": "..."}` envelope and the shared frontend API client (`api/client.ts`) unwraps it into a single readable message. The groups below are summarized from the Backend sections of the corresponding info docs under `resmon_reports/info_docs/`.

`/api/health`

- `GET /api/health` — cheap readiness probe used by the Electron main process to decide when to open the renderer window. Returns a small static JSON payload.

`/api/search`

Manual execution dispatch. Each endpoint registers a new execution with the admission controller, starts the `SweepEngine` pipeline on a worker thread, and returns the assigned execution id.

- `POST /api/search/dive` — single-repository Deep Dive. Body carries `repository`, `keywords`, optional `date_range`, `max_results`, `ai_enabled`, per-execution `ai_overrides`, and an optional ephemeral API key.
- `POST /api/search/sweep` — multi-repository Deep Sweep. Body carries `repositories` (array of slugs), `keywords`, optional `date_range`, `max_results` (applied **per repository**), `ai_enabled`, per-execution `ai_overrides`, and a map of ephemeral per-repository keys. Returns HTTP `429` with `Retry-After: 5` when the admission controller has no free slot.
- `GET /api/search/repositories` — repository catalog surface consumed by the search pages to render the repository selector.

`/api/routines`

CRUD, activation, cloud migration, and cancel control for scheduled Automated Deep Sweeps.

- `GET /api/routines` — list of local routines; used by the Routines and Calendar pages.
- `POST /api/routines` — create a local routine; the response includes the row's `id`, which is then stored as `linked_routine_id` inside the matching `routine` configuration row.
- `PUT /api/routines/{id}` — patch routine fields (including the inline Email / AI / Notify column toggles on the Routines page, which each patch a single flag).
- `DELETE /api/routines/{id}` — delete a routine; also removes the APScheduler job.
- `POST /api/routines/{id}/activate` / `POST /api/routines/{id}/deactivate` — register or unregister the APScheduler job without deleting the DB row. Invoked from the Routines table and from the Calendar popover's routine toggle.
- Cloud migration endpoints perform a destination-first create followed by a source delete so the **Move to Cloud** / **Move to Local** buttons on the routines table are safe to re-run on transient failures.

`/api/executions`

History, reporting, log and progress surfaces, export, delete, and cancel control for every execution (Deep Dive, Deep Sweep, Automated Deep Sweep).

- `GET /api/executions/merged?filter=<all|local|cloud>&limit=200` — unified local + cloud execution ledger consumed by the Results & Logs page's `useExecutionsMerged` hook.
- `GET /api/executions/{id}` — execution metadata row (type, status, query, counts, timestamps, routine id, etc.).
- `GET /api/executions/{id}/report` — rendered Markdown report.
- `GET /api/executions/{id}/log` — raw per-execution log text (written by `TaskLogger`).
- `GET /api/executions/{id}/progress/events` — historical progress event list. Returns the live `progress_store` events if the execution is still registered in memory and falls back to persisted events from the `execution_progress` table after cleanup.
- `GET /api/executions/{id}/progress/stream` — SSE progress stream (used by `ExecutionContext` and the Monitor page) with `~300 ms` heartbeats, `last_event_id` resumption, and a post-terminal drain.
- `GET /api/executions/active` — cheap `{active_ids: [...]}` payload polled every 3 seconds as a safety net that attaches background-initiated runs.
- `POST /api/executions/export` — body `{ "ids": [...] }`; the backend assembles a zip bundle of the selected executions' reports, logs, and artifacts and returns `{ "path": "<absolute path>" }`. When the Storage tab's `export_directory` is set, the zip lands there; otherwise a temp file is used.
- `POST /api/executions/{id}/cancel` — cooperative cancel: sets the `ProgressStore` cancel flag, the pipeline's 2-second heartbeat observes it, a partial report is flushed, and the execution finalizes as `cancelled`.
- `DELETE /api/executions/{id}` — delete a local execution row and its artifacts. Cloud rows are read-only from the Results & Logs page.

`/api/configurations`

See Configuration for the full round-trip workflow.

- `GET /api/configurations` — list all rows (optional `config_type` filter). `parameters` is JSON-decoded into a dict per row when possible, otherwise returned as the raw string.
- `POST /api/configurations` — create a configuration from a `ConfigCreate` body (`name`, `config_type`, `parameters`). Response: `{ "id", "name", "config_type" }`.
- `PUT /api/configurations/{id}` — update a configuration from a `ConfigUpdate` body (`name?`, `parameters?`); 404 if the id is unknown.
- `DELETE /api/configurations/{id}` — delete a configuration; cascades to the linked routine when `config_type == 'routine'` and the parameters carry a valid `linked_routine_id`.
- `POST /api/configurations/export` — body `{ "ids": [...] }`; writes a ZIP archive of per-config JSON files and returns `{ "path": "<absolute path>" }`.
- `POST /api/configurations/import` — accepts a multipart `files: list[UploadFile]` of `.json` files; response `{ "imported": <n>, "errors": [...] }`.

/api/calendar

- `GET /api/calendar/events?start=<iso>&end=<iso>` — combined payload of historical executions (from `get_executions(conn, limit=500)`) and expanded upcoming routine fires. `start` / `end` are parsed with `datetime.fromisoformat(...)` (trailing `Z` replaced with `+00:00`). Defaults clamp the window to `[now, now + 90 days]` ; `window_start` is forced to `now` so past fires are never synthesized from cron. Each event carries `id` , `title` , `start` , `end` , `color` , `execution_id` , `routine_id` , `type` , `status` , `query` , `total_results` , `new_results` . Scheduled expansion is capped at `MAX_PER_ROUTINE = 200` fires per routine per request.

/api/settings

Read/write surface for the eight Settings panels. Each slice is a keyed subset of the `settings` table, written through `set_setting` .

- `GET /api/settings/email` / `PUT /api/settings/email` — `smtp_server` , `smtp_port` , `smtp_username` , `smtp_from` , `smtp_to` .
- `POST /api/settings/email/test` — sends a test email using the saved SMTP settings and the stored `smtp_password` credential.
- `GET /api/settings/ai` / `PUT /api/settings/ai` — `ai_provider` , `ai_model` , `ai_local_model` , `ai_summary_length` , `ai_tone` , `ai_temperature` , `ai_extraction_goals` , `ai_custom_base_url` , `ai_custom_header_prefix` .
- `GET /api/settings/cloud` / `PUT /api/settings/cloud` — `cloud_auto_backup` and related toggles.
- `GET /api/settings/storage` / `PUT /api/settings/storage` — `pdf_policy` , `txt_policy` , `archive_after_days` , `export_directory` (policy values are constrained to `save` / `archive` / `discard`).
- `GET /api/settings/notifications` / `PUT /api/settings/notifications` — desktop-notification toggles.
- `GET /api/settings/execution` / `PUT /api/settings/execution` — admission-controller tunables (`max_concurrent` clamped to `[1, 8]` , routine queue clamped to `[1, 64]`); reloaded live by the admission controller.
- Diagnostics: `GET /api/scheduler/jobs` (APScheduler job snapshot consumed by Settings → Advanced), `POST /api/service/install` and related `/api/service/*` routes for OS-service integration via `service_manager.py` .

/api/credentials

Presence-only surface over the OS keyring through `credential_manager.py` . Secrets are never returned; `GET` responses only report whether a named credential exists.

- `GET /api/credentials` — map of `{credential_name: present_boolean}`.
- `PUT /api/credentials/{name}` — store a credential (Email panel uses `smtp_password`; AI panel derives names from the provider, e.g. `openai_api_key`, `anthropic_api_key`, `custom_llm_api_key`).
- `DELETE /api/credentials/{name}` — remove a credential.
- `POST /api/credentials/validate` — validate a remote LLM key (Test key button on the AI panel).
- `POST /api/ai/models` — list the per-provider model catalog using either the freshly typed key or the stored credential (Load models button on the AI panel).

`/api/cloud` and `/api/cloud-auth`

Two distinct surfaces — see Cloud Backup and Cloud Account for the distinction.

- `GET /api/cloud/status` — `{ is_linked, api_ok, api_reason }` describing the Google Drive link state.
- `POST /api/cloud/link` — triggers the Google Drive OAuth installed-app flow; the resulting token is stored in the OS keyring.
- `POST /api/cloud/unlink` — removes the stored token.
- `POST /api/cloud/backup` — ad-hoc backup of the report tree to the linked Drive folder; returns `{ uploaded, total_files, folder_name, web_view_link }`.
- `/api/cloud-auth/*` — resmon-cloud identity routes (sign-in, refresh, sign-out). Currently an "under construction" placeholder in the Cloud Account panel because no hosted identity provider is wired in this build.

AI Summarization

AI summarization is optional and fully bring-your-own-key. When the provider is unset, every pipeline call is a silent no-op and the Markdown report falls back to plain abstract extraction. Settings live under **Settings** → **AI** (`#/settings/ai`) and are persisted through `PUT /api/settings/ai`; the API key for the active provider is stored separately in the OS keyring through `PUT /api/credentials/{name}`.

Provider Whitelist

The AI panel enforces the following provider whitelist (IMPL-AI5 / AI9 / AI10); each provider has a suggested model placeholder for the `ai_model` field:

`ai_provider`	Kind	Credential name	Suggested model placeholder
`openai`	Remote (BYOK)	`openai_api_key`	`gpt-4o-mini`

`ai_provider`	Kind	Credential name	Suggested model placeholder
`anthropic`	Remote (BYOK)	`anthropic_api_key`	`claude-3-5-haiku-latest`
`google`	Remote (BYOK)	`google_api_key`	`gemini-2.5-flash`
`xai`	Remote (BYOK)	`xai_api_key`	`grok-4`
`meta`	Remote (BYOK)	`meta_api_key`	`meta-llama/Llama-3.3-70B-Instruct-Turbo`
`deepseek`	Remote (BYOK)	`deepseek_api_key`	`deepseek-chat`
`alibaba`	Remote (BYOK)	`alibaba_api_key`	`qwen-plus`
`local`	Local (ollama)	—	`llama3` (set through `ai_local_model`)
`custom`	Remote (BYOK, user-defined HTTPS endpoint)	`custom_llm_api_key`	user-specified

Configuring a Remote Provider (OpenAI, Anthropic, and the rest)

1. Open **Settings** → **AI**.
2. Select a provider from the **Provider** dropdown (e.g. `openai` or `anthropic`). The **Model** placeholder updates to the provider's suggested id.
3. Enter the model id in **Model** (free text; the **Load models** button can populate a picker from the provider's catalog via `POST /api/ai/models`).
4. Enter the API key in the password field; press **Store key** to write it to the OS keyring as `<provider>_api_key`. The key is never returned to the frontend — subsequent loads only see a presence boolean from `GET /api/credentials`.
5. Click **Test key** to issue `POST /api/credentials/validate`, which performs a minimal round-trip against the provider.

6. Tune `ai_summary_length`, `ai_tone`, `ai_temperature`, and `ai_extraction_goals` to shape the prompt scaffolding in `prompt_templates.py`.
7. Click **Save** (`PUT /api/settings/ai`).

Configuring the Custom Provider

The `custom` provider targets any HTTPS endpoint that accepts an OpenAI-compatible chat-completions request shape. Two extra fields apply:

- `ai_custom_base_url` — the endpoint root. The Save button is disabled unless the URL is HTTPS or the host is loopback (`localhost`, `127.0.0.1`, `::1`), enforced both by the frontend guard `validateCustomBaseUrl` and by the backend `llm_factory` as a hard check.
- `ai_custom_header_prefix` — the auth-header prefix applied to the stored `custom_llm_api_key` credential (e.g. `Bearer`).

Configuring the Local Provider (ollama)

The `local` provider bypasses the key flow entirely and dispatches through `implementation_scripts/llm_local.py` against a local ollama daemon over its REST API (`/api/generate`, `/api/tags`).

1. Install ollama and start it (`ollama serve`).
2. Pull at least one model (for example `ollama pull llama3`, `ollama pull gemma3`, or `ollama pull qwen2`).
3. In **Settings** → **AI**, set **Provider** to `local`.
4. Enter the model tag (e.g. `llama3`) in the **Local model** field — this writes to the separate `ai_local_model` key introduced by IMPL-AI11 so the model id cannot drift into `ai_tone`. A one-shot migration heuristic (`looksLikeModelId`) moves a misplaced value from `ai_tone` into `ai_local_model` on first load.
5. Click **Load models** to pull the available-models list from the running ollama instance, or enter the tag manually.
6. Click **Save**.

No API key is required for `local`; `Test key` and `Store key` are hidden for this provider.

Per-Execution Overrides

Deep Dive and Deep Sweep expose an **Override AI settings for this run** disclosure that accepts per-execution overrides for length, tone, and model. Empty override fields fall through to the persisted `ai_*` settings. Overrides flow through the request body of `POST /api/search/dive` / `POST /api/search/sweep` and are consumed by `SweepEngine` for that one execution only.

Email Notifications

Email notifications are transactional SMTP messages emitted by `email_notifier.py` + `email_sender.py` on routine completion. They are opt-in at two levels: the global SMTP configuration lives under **Settings** → **Email**, and each routine owns its own per-routine **Email** and **Results-in-Email** toggles that gate whether that specific routine notifies on completion and whether the AI summary is inlined in the body.

Configuring SMTP

Open **Settings** → **Email** (`#/settings/email`) and fill in the transport fields:

Field	Purpose
`smtp_server`	Outbound SMTP host (e.g. `smtp.gmail.com` , `smtp.office365.com`).
`smtp_port`	Port (`587` for STARTTLS, `465` for implicit TLS, `25` for plain).
`smtp_username`	Authenticating username.
`smtp_from`	`From:` address on outgoing notifications.
`smtp_to`	Default recipient address used by routines.

Press **Save** to persist the transport fields through `PUT /api/settings/email` .

Storing the SMTP Password

The SMTP password is stored **separately** from the transport fields, in the OS keychain under the credential name `smtp_password` :

1. Type the password into the password field.
2. Click **Store password**. The frontend strips whitespace from the input before sending (so a Gmail App Password — which Google's UI presents as four space-separated groups — becomes the raw 16-character secret) and issues `PUT /api/credentials/smtp_password` .
3. The UI afterwards sees only a presence boolean from `GET /api/credentials` ; the password itself is never returned.
4. Use **Remove password** (`DELETE /api/credentials/smtp_password`) to clear it.

Testing the Configuration

Click **Send test email** to issue `POST /api/settings/email/test` . The backend composes a minimal test message, binds to the stored SMTP server, authenticates with the stored credential, and returns a

success or `Error:` status line that is surfaced inline in the panel.

Per-Routine Controls

When SMTP is configured, a routine emits notifications only when its **Email** toggle is on. Two additional per-routine flags refine the behavior:

- **Results-in-Email** — inlines the AI summary inside the email body.
- **Notify-on-Completion** — independent desktop notification raised on fire completion; it does not depend on SMTP.

The execution bundle produced by the shared export pipeline (`/api/executions/export`) is attached to the email when the total size is within the configured limit.

Cloud Backup and Cloud Account

resmon exposes two distinct cloud surfaces that must not be conflated. They live under separate Settings panels and use separate credential stores.

1. **Cloud Storage** — Google Drive artifact backup scoped to the report tree. Least-privilege, user-supplied OAuth client, no hosted microservice.
2. **Cloud Account** — optional resmon-cloud identity and hybrid execution model. Enables cloud-executed routines and the merged local/cloud executions view on the Results & Logs page.

Cloud Storage — Linking Google Drive

This surface is a thin wrapper over the Drive v3 API using the least-privilege `drive.file` OAuth 2.0 scope, meaning resmon can only see files it created itself. The OAuth client secrets are user-supplied and live in `credentials.json` at the project root (gitignored).

1. Create an OAuth 2.0 client of type **Desktop app** in Google Cloud Console, enable the Drive API, and download the client secrets JSON to the project root as `credentials.json` .
2. Open **Settings** → **Cloud Storage** (`#/settings/cloud`).
3. Click **Link Google Drive**. The frontend issues `POST /api/cloud/link` ; the backend runs the OAuth installed-app flow (`google-auth-oauthlib`), prompts for consent in the default browser, receives the access + refresh tokens, and stores them in the OS keyring. `GET /api/cloud/status` then returns `{ is_linked: true, api_ok: true }` .
4. Toggle **Auto-backup** to persist `cloud_auto_backup` through `PUT /api/settings/cloud` . When on, `SweepEngine._maybe_auto_backup` uploads the execution bundle to the linked Drive folder at the end of every fire.

5. Click **Back up now** (`POST /api/cloud/backup`) for an ad-hoc backup. The response `{ uploaded, total_files, folder_name, web_view_link }` powers the banner and the "open in Drive" link.
6. Click **Unlink** (`POST /api/cloud/unlink`) to revoke the stored token.

Drive API errors are surfaced with targeted hints via `API_REASON_HINTS` for `accessNotConfigured` , `insufficientPermissions` , and `no_token` , so the user can resolve Google Cloud Console / scope-consent issues directly from the panel.

Cloud Account — resmon-cloud Sign-In and Hybrid Execution

The resmon-cloud surface is a separate microservice (sources under `resmon_scripts/cloud/`) that backs envelope-encrypted per-user credentials (per-user DEK wrapped by a KMS-held KEK), JWKS-verified JWTs, row-level security, signed-URL artifact fetches, and per-user rate limiting. The desktop consumes it through `AuthContext` , `api/cloudClient.ts` , and the `useCloudSync` hook, which polls `GET /api/v2/sync?since=<version>` every 60 seconds (also on focus-gain), drains `has_more` chains up to `MAX_PAGES_PER_TICK = 50` , and POSTs pages into the local daemon's `/api/cloud-sync/ingest` .

Current build status: the **Cloud Account** panel (`#/settings/account`) is an "under construction" placeholder. The `/api/cloud-auth/*` routes exist in the backend but no hosted identity provider is wired in this build, so the panel renders a fixed `PageHelp` block describing the intent — cloud routines, cloud-scoped credentials, cloud-executed reports, and the privacy invariant that local executions never depend on cloud sign-in — and no sign-in control is available.

When sign-in is enabled in a future build, three surfaces on the desktop become active:

- The **Routines** page's create/edit modal unlocks the **Cloud** execution-location radio so a routine can fire on the `resmon-cloud` scheduler instead of the local APScheduler thread.
- The **Results & Logs** page's `useExecutionsMerged('all', 200)` hook exposes the Local / Cloud / All filter chip and renders cloud rows as read-only entries with a cached-artifacts informational card. Cloud artifacts are fetched on demand via signed URLs and cached locally under `~/Library/Application Support/resmon/cloud-cache/` .
- The **Move to Cloud** / **Move to Local** buttons on the routines table perform a destination-first create followed by a source delete, safe to retry on transient failures.

Until the hosted identity provider is live, all routines run locally on the APScheduler thread and the Results & Logs page shows only local executions. No credentials or data leave the machine unless the user explicitly links Google Drive (Cloud Storage) or signs in to resmon-cloud (Cloud Account).

License

resmon is released under the **MIT License**. Copyright (c) 2026 Ryan Kamp. The full license text is available in LICENSE at the project root.

In short, the MIT License permits use, copying, modification, merging, publication, distribution, sublicensing, and sale of the software, subject to the condition that the copyright notice and permission notice be included in all copies or substantial portions of the software. The software is provided "as is," without warranty of any kind.

Contributing

Contributions are welcome. resmon is a single-maintainer project, so the workflow below is deliberately lightweight; please open an issue before starting substantial work so the scope can be scoped and any related changes in the `.ai:/prep/` planning documents can be coordinated.

Reporting Issues

Open a GitHub issue with:

1. A short, specific title.
2. The resmon version (git commit hash or packaged build number), host OS, Python version (`python3 --version`), and Node.js version (`node --version`).
3. Reproduction steps — the page, the operation, the exact form input, and the observed error.
4. Relevant log excerpts. Per-execution logs live under `resmon_reports/logs/` and the rotating app log is `resmon_reports/logs/resmon.log` . Redact any API keys or personal data before pasting.
5. A screenshot of the UI state when the bug is visual.

Security-sensitive reports (credential handling, SQL injection, OAuth flow, cloud sync, keyring access) should not be filed as public issues. Email the maintainer directly instead.

Submitting Pull Requests

1. Fork the repository and create a topic branch off `main` (`feat/<short-slug>` , `fix/<short-slug>` , or `docs/<short-slug>`).
2. Keep the change focused — one logical unit per pull request.
3. Match the existing code style. Python follows PEP 8 with 4-space indentation; TypeScript/TSX follows the repository's Prettier defaults. Do not reformat unrelated files.
4. Run the verification suites before pushing:
   - Backend: `pytest resmon_scripts/verification_scripts/`
   - Frontend type check: `cd resmon_scripts/frontend && npm run typecheck`

- Frontend renderer tests: `cd resmon_scripts/frontend && npm test`

5. If the change touches a repository client, add or update the corresponding test under `resmon_scripts/verification_scripts/` and the matching fixture. New repositories additionally require a row in `repo_catalog.py` and a registration in `api_registry.py`.
6. If the change alters user-visible behavior, update the affected page info document under `resmon_reports/info_docs/` and, where relevant, this README.
7. Open the pull request against `main`. The description must explain **what** changed, **why** it changed, and how the change was verified. Link the originating issue.

Commit Messages

Commit messages follow a concise, imperative style (`Add IEEE Xplore rate-limit fallback` , not `Added` or `Adds`). Multi-line bodies are encouraged for non-trivial changes; reference the issue number in the body rather than the subject line.

Code of Conduct

Be respectful and constructive. Focus feedback on the code and the technical trade-offs, not the contributor.

Acknowledgments

resmon is built on top of a broad ecosystem of open-access scholarly repositories and open-source software. The project gratefully acknowledges:

Open-Access Repository Providers

The 16 scholarly sources registered in the repository catalog, whose public APIs make automated literature surveillance possible:

- **arXiv** — Cornell University / arXiv.org, for the Atom XML API and the decades-long commitment to open preprint distribution in physics, mathematics, computer science, quantitative biology, statistics, electrical engineering, and economics.
- **bioRxiv** and **medRxiv** — Cold Spring Harbor Laboratory, for the shared preprint JSON API serving the life- and health-sciences communities.
- **CORE** — The Open University / Jisc, for the aggregated open-access JSON API spanning tens of thousands of repositories worldwide.
- **CrossRef** — Crossref, for the DOI-indexed REST API and the "polite pool" that rewards well-behaved clients with priority rate limits.
- **DBLP** — Schloss Dagstuhl / University of Trier, for the computer-science bibliography REST API.

- **DOAJ** — Directory of Open Access Journals / Infrastructure Services for Open Access C.I.C., for the journal- and article-level JSON API covering open-access journals across disciplines.
- **EuropePMC** — EMBL-EBI on behalf of the Europe PMC Consortium, for the biomedical and life-sciences REST API.
- **HAL** — CCSD / CNRS, for the Solr-backed multi-disciplinary JSON API.
- **IEEE Xplore** — IEEE, for the Xplore REST API serving electrical engineering, computer science, and electronics literature.
- **NASA ADS** — Smithsonian Astrophysical Observatory / NASA Astrophysics Data System, for the Solr-backed astronomy, astrophysics, and planetary-science API.
- **OpenAlex** — OurResearch, for the free, comprehensive scholarly-works REST API and the mailto-based polite-pool rate tier.
- **PLOS** — Public Library of Science, for the Solr-backed JSON API over the PLOS journal family.
- **PubMed / NCBI E-utilities** — U.S. National Library of Medicine / National Center for Biotechnology Information, for the E-utilities suite that underpins biomedical literature retrieval.
- **Semantic Scholar** — Allen Institute for AI (AI2), for the cross-disciplinary scholarly-graph REST API.
- **Springer Nature** — Springer Nature, for the Meta API covering STM, humanities, and social-sciences content.

Open-Source Foundations

resmon depends on and is grateful for the following open-source projects (non-exhaustive): **Python**, **FastAPI**, **Starlette**, **Uvicorn**, **Pydantic**, **httpx**, **lxml**, **BeautifulSoup**, **SQLAlchemy**, **APScheduler**, **cryptography**, **keyring**, **NLTK**, **tiktoken**, **pytest**, **Electron**, **Node.js**, **React**, **React Router**, **TypeScript**, **Webpack**, **Tailwind CSS**, **FullCalendar**, **electron-builder**, and **ollama**. The maintainers and contributors of these projects make a desktop-class literature surveillance tool buildable by a single developer.

Standards and Identifiers

resmon relies on the **DOI** system administered by the International DOI Foundation and the **ORCID** identifier system — both of which underpin the deduplication and citation-graphing pipeline.